OBJECT-ORIENTED PROGRAMMING (OOP) — COMPLETE TVET NOTES

Topic 1: Introduction to Object-Oriented Programming

Topic 2: Classes and Objects

Topic 3: Encapsulation

Topic 4: Constructors and Destructors

Topic 5: Inheritance

Topic 6: Polymorphism

Topic 7: Abstraction

Topic 8: Access Specifiers

Topic 9: Functions and Methods in OOP

Topic 10: Arrays and Objects

Topic 11: Relationships Between Classes

Topic 12: Exception Handling

Topic 13: File Handling in OOP

Topic 14: OOP Practical Projects and Revision

WHATSAPP:0720349953

TOPIC 1: INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING

1.1 What is Programming?

Programming is the process of creating instructions that tell a computer how to perform a particular task.

Examples of programming languages include:

C

C++

Python

Java

C#

JavaScript

1.2 What is Object-Oriented Programming?

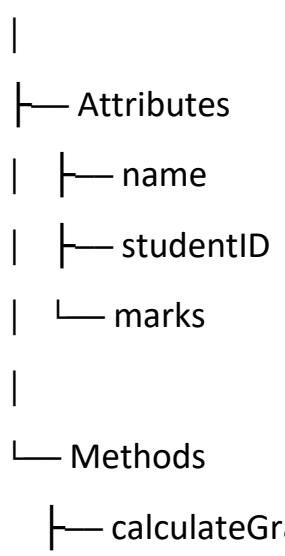
Object-Oriented Programming (OOP) is a programming approach that organizes software around objects and classes rather than only functions and procedures.

An object represents something that has:

Data/attributes

Behavior/methods

For example, a student can be represented as an object.



└── displayDetails()

1.3 Examples of Objects

Objects can represent real-world entities such as:

Student

Bank account

Employee

Car

Phone

Customer

Product

Book

1.4 Main Principles of OOP

The four major principles are:

1. Encapsulation

Combining data and methods into one unit and controlling access to the data.

2. Inheritance

Allowing one class to acquire properties and behaviors from another class.

3. Polymorphism

Allowing the same interface or operation to behave differently in different situations.

4. Abstraction

Hiding unnecessary implementation details and exposing only essential functionality.

1.5 Advantages of OOP

Code reusability

Easier maintenance

Better organization

Data security

Modularity

Easier debugging

Supports large applications

Reduces code duplication

1.6 OOP vs Procedural Programming

Procedural Programming	OOP
Focuses on functions	Focuses on objects
Data and functions are often separate	Data and methods are combined
Less emphasis on data protection	Supports data hiding

Code reuse can be limited Strong code reuse through inheritance

Example: C Examples: C++, Java, C#

Revision Questions

Define OOP.

What is an object?

State the four principles of OOP.

Give five examples of objects.

State five advantages of OOP.

Differentiate between procedural programming and OOP.

TOPIC 2: CLASSES AND OBJECTS

2.1 What is a Class?

A class is a blueprint or template used to create objects.

For example:

```
class Student {
public:
   string name;
   int age;
};
```

The class defines what information a student object can contain.

2.2 What is an Object?

An object is an instance of a class.

Example:

```
Student student1;
Student student2;
```

Here:

Student is the class.
student1 is an object.
student2 is an object.

2.3 Basic Class Structure

```
class Student {
public:
    string name;
    int age;

    void display() {
        cout << name << endl;
        cout << age << endl;
    }
```

```
};
```

Creating an object:

```
Student student1;
```

Assigning values:

```
student1.name = "Joseph";
student1.age = 20;
```

Calling a method:

```
student1.display();
```

2.4 Complete Example

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int age;

    void display() {
```

```
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

int main() {

    Student student1;

    student1.name = "Joseph";
    student1.age = 20;

    student1.display();

    return 0;
}
```

Output

```
Name: Joseph
Age: 20
```

2.5 Attributes

Attributes are variables that describe an object.

Example:

```
class Car {
public:
   string brand;
   string color;
   int year;
};
```

Attributes:

brand

color

year

2.6 Methods

Methods are functions defined inside a class.

```
class Car {
public:

   void start() {
      cout << "Car started";
   }
};
```

start() is a method.

Revision Questions

Define a class.

Define an object.

Differentiate between a class and an object.

What are attributes?

What are methods?

Write a C++ class called Employee.

TOPIC 3: ENCAPSULATION

3.1 What is Encapsulation?

Encapsulation is the process of combining data and methods into a single unit and controlling access to the data.

A class provides a natural way to achieve encapsulation.

3.2 Why Encapsulation is Important

It helps:

Protect data

Prevent unauthorized modification

Control access

Improve security

Make code easier to maintain

3.3 Private Data

Example:

```
class BankAccount {

private:
   double balance;

public:

   void deposit(double amount) {
      balance += amount;
   }
};
```

The balance variable cannot be directly accessed from outside the class.

3.4 Getters and Setters

A getter retrieves a value.

A setter changes a value.

Example:

```
class Student {

private:
   int age;

public:

   void setAge(int a) {
      age = a;
   }

   int getAge() {
      return age;
   }
};
```

Using the class:

```
Student student;
```

```
student.setAge(20);

cout << student.getAge();
```

3.5 Data Hiding

Data hiding prevents direct access to important internal data.

Example:

```
class Employee {

private:
   double salary;

public:

   void setSalary(double s) {
      salary = s;
   }

   double getSalary() {
      return salary;
   }
};
```

The salary is protected from direct modification.

Revision Questions

Define encapsulation.

What is data hiding?

What is a getter?

What is a setter?

Why is encapsulation important?

Write a class containing a private balance variable and getter/setter methods.

TOPIC 4: CONSTRUCTORS AND DESTRUCTORS

4.1 Constructor

A constructor is a special member function that is automatically called when an object is created.

It has the same name as the class.

Example:

```
class Student {

public:

    Student() {
```

```
        cout << "Student created";
    }
};
```

Creating an object:

```
Student student1;
```

The constructor automatically executes.

4.2 Default Constructor

A constructor that does not require parameters.

```
class Student {

public:

    Student() {
        cout << "Default constructor";
    }
};
```

4.3 Parameterized Constructor

A constructor that receives arguments.

```
class Student {

public:

   string name;
   int age;

   Student(string n, int a) {
      name = n;
      age = a;
   }
};
```

Creating an object:

```
Student student1("Joseph", 20);
```

4.4 Constructor Example

```
#include <iostream>
using namespace std;

class BankAccount {
```

```
private:
    string name;
    double balance;

public:

    BankAccount(string n, double b) {
        name = n;
        balance = b;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Balance: " << balance << endl;
    }
};

int main() {

    BankAccount account("Joseph", 50000);

    account.display();

    return 0;
```

```
}
```

4.5 Destructor

A destructor is a special function that is automatically called when an object is destroyed.

It uses a tilde ~.

```
class Student {

public:

   ~Student() {
      cout << "Object destroyed";
   }
};
```

4.6 Constructor vs Destructor

Constructor	Destructor
Initializes object	Cleans up object
Same name as class	Same name preceded by ~
Called when object is created	Called when object is destroyed
Can have parameters	Cannot have parameters
Can be overloaded	Cannot normally be overloaded

Revision Questions

Define a constructor.

What is a default constructor?

What is a parameterized constructor?

Define a destructor.

Differentiate between a constructor and destructor.

Write a C++ class with a parameterized constructor.

TOPIC 5: INHERITANCE

5.1 What is Inheritance?

Inheritance is an OOP mechanism where one class acquires properties and methods from another class.

The existing class is called the:

Base/Parent/Superclass

The new class is called the:

Derived/Child/Subclass

5.2 Example

```
class Animal {

public:

    void eat() {
```

```
        cout << "Animal is eating";
    }
};

class Dog : public Animal {

public:

    void bark() {
        cout << "Dog is barking";
    }
};
```

Using it:

```
Dog dog;

dog.eat();
dog.bark();
```

The Dog class inherits eat() from Animal.

5.3 Types of Inheritance

1. Single Inheritance

One child inherits from one parent.

A

↓

B

2. Multilevel Inheritance

Inheritance occurs in multiple levels.

A

↓

B

↓

C

Example:

```
class Animal {};
class Dog : public Animal {};
class Puppy : public Dog {};
```

3. Multiple Inheritance

One class inherits from multiple parent classes.

A B

\ /

  C

Example:

```
class A {};
class B {};

class C : public A, public B {};
```

4. Hierarchical Inheritance

Multiple child classes inherit from one parent.

```
     Animal
     /   \
   Dog   Cat
```

5. Hybrid Inheritance

A combination of two or more types of inheritance.

5.4 Advantages of Inheritance

Code reuse

Reduces duplication

Easier maintenance

Supports hierarchical classification

Improves program organization

Revision Questions

Define inheritance.

What is a base class?

What is a derived class?

Explain single inheritance.

Explain multilevel inheritance.

Explain multiple inheritance.

State four advantages of inheritance.

TOPIC 6: POLYMORPHISM

6.1 What is Polymorphism?

Polymorphism means "many forms."

It allows the same method, function or interface to behave differently depending on the situation.

6.2 Types of Polymorphism

Two important forms are:

Compile-time polymorphism

Runtime polymorphism

6.3 Function Overloading

Function overloading occurs when multiple functions have the same name but different parameters.

```
class Calculator {

public:

   int add(int a, int b) {
      return a + b;
   }

   int add(int a, int b, int c) {
      return a + b + c;
   }
};
```

The same function name add() performs different operations based on the arguments.

6.4 Operator Overloading

C++ allows operators to be given special meanings for user-defined types.

Example:

operator+

can be overloaded for a class.

6.5 Function Overriding

A derived class can provide its own implementation of a method inherited from a base class.

```
class Animal {

public:

   virtual void sound() {
      cout << "Animal sound";
   }
};

class Dog : public Animal {

public:
```

```
    void sound() override {
        cout << "Dog barks";
    }
};
```

6.6 Virtual Functions

A virtual function allows the appropriate overridden method to be selected at runtime when accessed through a base-class reference or pointer.

Revision Questions

Define polymorphism.

What is function overloading?

What is function overriding?

What is a virtual function?

Differentiate between compile-time and runtime polymorphism.

TOPIC 7: ABSTRACTION

7.1 What is Abstraction?

Abstraction is the process of hiding unnecessary implementation details and exposing only the important features of an object.

Example:

When using an ATM, you see:

1. Check Balance

2. Withdraw

3. Deposit

You do not need to know the internal banking software implementation.

7.2 Abstraction in C++

C++ supports abstraction through:

Classes

Access control

Abstract classes

Pure virtual functions

7.3 Abstract Class

An abstract class contains at least one pure virtual function.

```
class Animal {

public:

   virtual void sound() = 0;
};
```

The = 0 indicates a pure virtual function.

7.4 Derived Class

```
class Dog : public Animal {


public:


   void sound() override {
      cout << "Dog barks";
   }
};
```

7.5 Abstraction vs Encapsulation

Abstraction	Encapsulation
Hides implementation details	Controls access to data
Focuses on what an object does	Focuses on how data and methods are packaged/protected
Simplifies usage	Protects internal state

Revision Questions

Define abstraction.

Give a real-world example of abstraction.

What is an abstract class?

What is a pure virtual function?

Differentiate abstraction and encapsulation.

TOPIC 8: ACCESS SPECIFIERS

Access specifiers determine how class members can be accessed.

C++ provides:

public

private

protected

8.1 Public

Public members can be accessed from outside the class.

```
class Student {

public:
   string name;
};
```

Usage:

```
Student s;

s.name = "Joseph";
```

8.2 Private

Private members can only be accessed directly from within the class.

```
class Student {

private:
   int age;
};
```

This would not be allowed:

```
Student s;

s.age = 20;
```

8.3 Protected

Protected members can be accessed within the class and by derived classes.

```
class Parent {

protected:
   int value;
};
```

8.4 Comparison

Access	Class	Derived Class	Outside
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No direct access	No

Revision Questions

Name the three C++ access specifiers.

Explain public.

Explain private.

Explain protected.

Which access specifier provides the greatest restriction?

TOPIC 9: FUNCTIONS AND METHODS IN OOP

9.1 Member Function

A function declared inside a class is called a member function or method.

```
class Student {

public:

   void display() {
      cout << "Student details";
   }
};
```

9.2 Method With Parameters

```
class Calculator {

public:

   int add(int a, int b) {
      return a + b;
   }
};
```

9.3 Method Returning a Value

```
class Student {

public:

   int calculateTotal(int a, int b, int c) {
      return a + b + c;
   }
};
```

9.4 Const Member Function

A const member function promises not to modify the object's data.

```
class Student {

public:
```

```
    void display() const {
        cout << "Student";
    }
};
```

9.5 Static Members

A static data member belongs to the class rather than each individual object.

Example:

```
class Student {

public:

    static int count;
};
```

Revision Questions

Define a member function.

Write a method that adds two numbers.

What is a static member?

What is a return value?

Differentiate between a function with parameters and one without parameters.

TOPIC 10: ARRAYS AND OBJECTS

Objects can be stored in arrays.

Example:

```
class Student {

public:
   string name;
   int age;
};
```

Create an array:

```
Student students[3];
```

Assign values:

```
students[0].name = "Joseph";
students[0].age = 20;

students[1].name = "Joel";
students[1].age = 21;
```

```
students[2].name = "Mercy";
students[2].age = 19;

Display:

for(int i = 0; i < 3; i++) {

   cout << students[i].name << endl;
   cout << students[i].age << endl;
}
```

10.1 Practical Example: Student Management

```
#include <iostream>
using namespace std;

class Student {

public:

   string name;
   int marks;

   void display() {
      cout << "Name: " << name << endl;
      cout << "Marks: " << marks << endl;
```

```
    }
};

int main() {

    Student students[3];

    students[0].name = "Joseph";
    students[0].marks = 80;

    students[1].name = "Joel";
    students[1].marks = 75;

    students[2].name = "Mercy";
    students[2].marks = 90;

    for(int i = 0; i < 3; i++) {
        students[i].display();
        cout << endl;
    }

    return 0;
}
```

Revision Questions

Can an array contain objects?

Write a class called Employee.

Create an array containing five Employee objects.

Write a loop to display all employees.

TOPIC 11: RELATIONSHIPS BETWEEN CLASSES

Classes can interact with one another.

Important relationships include:

Association

A general relationship between two classes.

Example:

Teacher → teaches → Student

Aggregation

A "has-a" relationship where objects can exist independently.

Example:

Department

↓

Teachers

A teacher can exist even if the department is removed.

Composition

A stronger "has-a" relationship where one object owns another.

Example:

House

↓

Room

A room is treated as part of the house in the model.

Inheritance

An "is-a" relationship.

Dog IS-A Animal

11.1 IS-A vs HAS-A

IS-A

Represents inheritance.

Dog IS-A Animal

HAS-A

Represents composition or aggregation.

Car HAS-A Engine

Revision Questions

Define association.

Define aggregation.

Define composition.

What does IS-A mean?

What does HAS-A mean?

Give three examples of class relationships.

TOPIC 12: EXCEPTION HANDLING

12.1 What is an Exception?

An exception is an unexpected event that occurs while a program is running.

Examples:

Division by zero

Invalid input

File not found

Invalid array access

12.2 Exception Handling in C++

C++ uses:

try

throw

catch

12.3 Example

```
#include <iostream>
using namespace std;

int main() {

   try {

      int age;

      cout << "Enter age: ";
      cin >> age;

      if(age < 0) {
```

```
        throw age;
    }

    cout << "Age: " << age;
  }

  catch(int error) {

    cout << "Invalid age";
  }

  return 0;
}
```

12.4 Multiple Catch Blocks

Different types of exceptions can be handled separately.

```
try {

  // code

}
catch(int error) {
```

```
    // integer error

}
catch(string error) {

    // string error
}
```

12.5 Advantages of Exception Handling

Prevents sudden program termination

Improves reliability

Separates error handling from normal code

Makes programs easier to maintain

Revision Questions

Define an exception.

What are try, throw, and catch?

Give four examples of exceptions.

Explain why exception handling is important.

TOPIC 13: FILE HANDLING IN OOP

13.1 What is File Handling?

File handling allows a program to store and retrieve data from files.

C++ commonly uses:

ofstream

ifstream

fstream

13.2 Writing to a File

```
#include <fstream>
using namespace std;

int main() {

   ofstream file("students.txt");

   file << "Joseph\n";
   file << "Joel\n";

   file.close();

   return 0;
}
```

13.3 Reading From a File

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
```

```
    ifstream file("students.txt");

    string name;

    while(getline(file, name)) {
        cout << name << endl;
    }

    file.close();

    return 0;
}
```

13.4 File Modes

Common modes include:

Mode	Purpose
ios::in	Read
ios::out	Write
ios::app	Append
ios::binary	Binary mode

13.5 OOP File Example

```
class Student {
```

```
public:

    string name;
    int marks;

    void save() {

        ofstream file("students.txt");

        file << name << endl;
        file << marks << endl;

        file.close();
    }
};
```

Revision Questions

Define file handling.

What is ofstream?

What is ifstream?

What is fstream?

Explain ios::app.

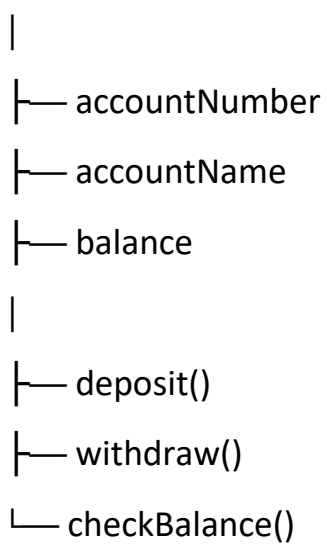
Write a program that saves student information to a file.

TOPIC 14: OOP PRACTICAL PROJECTS AND REVISION

PROJECT 1: BANK ACCOUNT SYSTEM

A simple OOP bank system can contain:

BankAccount



Example:

```
#include <iostream>
using namespace std;

class BankAccount {

private:

   string name;
```

```
    double balance;

public:

    BankAccount(string n, double b) {
        name = n;
        balance = b;
    }

    void deposit(double amount) {
        balance += amount;
    }

    void withdraw(double amount) {

        if(amount <= balance) {
            balance -= amount;
            cout << "Withdrawal successful\n";
        }
        else {
            cout << "Insufficient balance\n";
        }
    }
```

```
    void checkBalance() {
        cout << "Account Name: " << name << endl;
        cout << "Balance: " << balance << endl;
    }
};

int main() {

    BankAccount account("Joseph", 50000);

    account.deposit(5000);
    account.withdraw(10000);

    account.checkBalance();

    return 0;
}
```

This project demonstrates:

Class

Object

Constructor

Encapsulation

Private data

Public methods

Conditional statements

PROJECT 2: STUDENT MANAGEMENT SYSTEM

Possible features:

1. Add Student
2. View Student
3. Update Marks
4. Calculate Grade
5. Search Student
6. Delete Student
7. Exit

Class design:

Student

|

├── studentID

├── name

├── marks

├── grade

|

├── calculateGrade()

├── displayStudent()

└── updateMarks()

PROJECT 3: EMPLOYEE MANAGEMENT SYSTEM

Class:

Employee

|

├── employeeID

├── name

├── department

├── salary

|

├── calculateSalary()

├── displayEmployee()

└── updateEmployee()

PROJECT 4: LIBRARY MANAGEMENT SYSTEM

Possible classes:

Book

Member

Librarian

Library

Features:

Add book

Search book

Borrow book

Return book

Display books

Register member

PROJECT 5: ATM SYSTEM

Possible classes:

Account

ATM

Customer

Transaction

Features:

PIN authentication

Check balance

Deposit

Withdraw

Transfer

Exit

OOP REVISION QUESTIONS

Section A: Multiple Choice

1. What is a class?

A. An object

B. A blueprint for objects

C. A variable

D. A loop

Answer: B

2. Which concept protects data from direct access?

A. Inheritance

B. Encapsulation

C. Polymorphism

D. Compilation

Answer: B

3. Which keyword is used for inheritance in C++?

A. inherits

B. extends

C. :

D. inherit

Answer: C

4. Which function is automatically called when an object is created?

A. Destructor

B. Constructor

C. Main function

D. Static function

Answer: B

5. Which symbol identifies a destructor?

A. #

B. @

C. ~

D. $

Answer: C

6. Which access specifier provides the most restricted direct access?

A. public

B. protected

C. private

D. global

Answer: C

7. Polymorphism means:

A. One class only

B. Many forms

C. Hiding data

D. Creating files

Answer: B

8. Which keyword is associated with runtime polymorphism in C++?

A. virtual

B. static

C. private

D. friend

Answer: A

9. Which concept means hiding unnecessary implementation details?

A. Inheritance

B. Abstraction

C. Encapsulation

D. Compilation

Answer: B

10. Which C++ object is commonly used to write to a file?

A. ifstream

B. ofstream

C. cin

D. cout

Answer: B

SECTION B: SHORT-ANSWER QUESTIONS

Define Object-Oriented Programming.

Define a class.

Define an object.

Explain encapsulation.

Explain inheritance.

Explain polymorphism.

Explain abstraction.

What is a constructor?

What is a destructor?

Differentiate between public and private.

What is function overloading?

What is function overriding?

What is a virtual function?

What is an abstract class?

Explain data hiding.

What is a getter?

What is a setter?

Explain association.

Explain aggregation.

Explain composition.

SECTION C: PRACTICAL QUESTIONS

Question 1

Create a C++ class called Student with:

Name

Student ID

Marks

Create methods to:

Enter student information

Calculate grade

Display student information

Question 2

Create a BankAccount class with:

Account number

Account name

Balance

Add methods for:

Deposit

Withdrawal

Balance inquiry

The system should prevent withdrawal when the balance is insufficient.

Question 3

Create a parent class called:

Person

with:

name

age

Create a child class:

Student

with:

studentID

course

Demonstrate inheritance.

Question 4

Create a Calculator class demonstrating function overloading.

It should support:

add(int, int)

add(int, int, int)

add(double, double)

OOP QUICK GLOSSARY

Term	Meaning
OOP	Object-Oriented Programming
Class	Blueprint for creating objects
Object	Instance of a class
Attribute	Data describing an object
Method	Function belonging to a class
Encapsulation	Combining data and methods while controlling access
Inheritance	Acquiring features from another class
Polymorphism	One interface/operation having different forms
Abstraction	Hiding unnecessary implementation details
Constructor	Initializes an object
Destructor	Performs cleanup when an object is destroyed
Parent class	Class being inherited from
Child class	Class that inherits

Overloading	Same name with different parameters
Overriding	Redefining an inherited method
Getter	Retrieves a value
Setter	Changes a value
public	Accessible from outside the class
private	Directly accessible only within the class
protected	Accessible within the class and derived classes
Virtual function	Supports dynamic dispatch
Object instance	A concrete object created from a class

FINAL OOP REVISION CHECKLIST

Before an OOP examination, a student should be able to:

- Define OOP
- Explain classes
- Explain objects
- Create a C++ class
- Create objects
- Create methods
- Use constructors
- Use destructors
- Explain encapsulation
- Implement getters and setters
- Explain data hiding

Explain inheritance

Implement single inheritance

Implement multilevel inheritance

Implement multiple inheritance

Explain polymorphism

Implement function overloading

Explain function overriding

Use virtual functions

Explain abstraction

Create abstract classes

Use access specifiers

Work with arrays of objects

Explain class relationships

Handle exceptions

Work with files

Build an OOP-based application

Debug basic OOP programs